



CHAPTER 8

Core JavaScript Concepts & Functions



Basics of JavaScript and ECMAScript (ES6)

Introduction to JavaScript

JavaScript is one of the **most widely used programming languages** for building **web applications**. It allows developers to combine **HTML and CSS** with programming logic to make web pages **interactive and dynamic**.

Key Features of JavaScript

- Enables **user interaction** with web applications
- Used for **animations**, pop-ups, form validation, and dynamic content
- Initially created to “**make web pages alive**”
- JavaScript programs are called **scripts**
- Scripts are written inside HTML and executed automatically when the page loads
- It is a **lightweight, interpreted, and object-oriented** language
- Can be used on both **client-side and server-side**
- Uses the **Browser DOM (Document Object Model)**
- JavaScript file extension is **.js**

Why is it Called JavaScript?

When JavaScript was created, it was first named **LiveScript**.

At that time, **Java** was extremely popular, so the language was renamed **JavaScript** for marketing purposes.

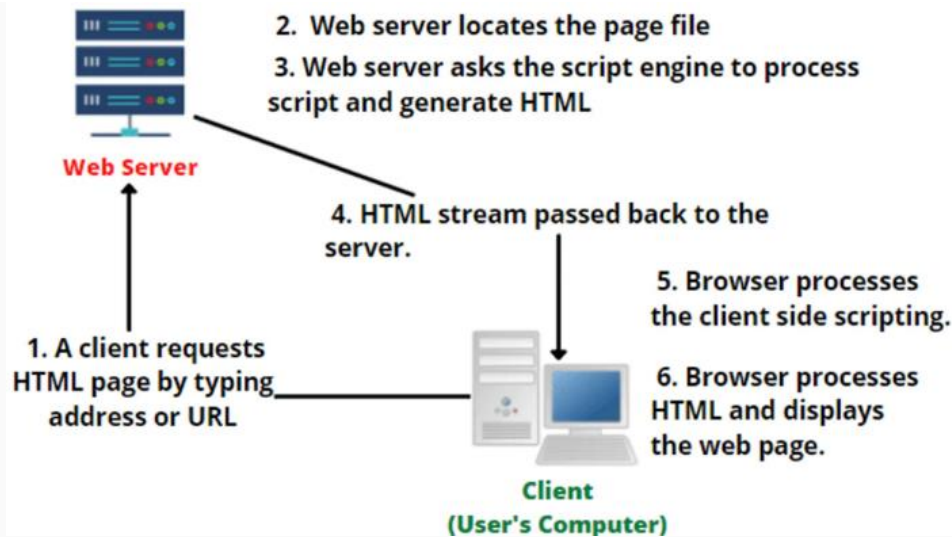
Where JavaScript is used

- Web development (frontend & backend)
- Mobile applications
- Desktop applications
- Game development
- Server-side programming

Client-side scripting

- ✓ Client-side scripting refers to programs that run on the **user's web browser** instead of the web server.
The script executes on the **client's computer** while the web page is loading or after it has loaded.

- ✓ It is mainly used to create **dynamic and interactive user interfaces** such as form validation, menus, animations, and navigation tools.
Client-side scripts can be written **inside HTML pages** or stored in **external script files**, which are sent from the server and executed by the browser.
- ✓ look at the diagram below.



Syntax of JavaScript

- ✓ JavaScript can be implemented using JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.
- ✓ You can place the **<script>** tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the **<head>** tags.
- ✓ The script tag takes two important attributes –
 - **Language** – This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
 - **Type** – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".
- ✓ The **<script>** tag alerts the browser program to start interpreting all the text between these tags as a script.

```
<script language = "javascript" type = "text/javascript">  
JavaScript code  
</script>
```

There are 2 ways to use the javascript in the HTML file:

Internal JavaScript: JavaScript can be added directly to the HTML file by writing the code inside the <script> tag . We can place the <script> tag either inside <head> or the <body> tag according to the need.

Example:

```
<html>
<head>
  <script language = "javascript" type = "text/javascript">
    document.write("Hello World!")
    console.log("Hello!")
  </script>
</head>
</html>
```

External JavaScript: The other way is to write JavaScript code in another file having a .js extension and then link the file inside the <head> or <body> tag of the HTML file in which we want to add this code.

Example:

```
j1.html
<html>
<head>
  <script src="j1.js"></script>
</head>
</html>
j1.js
document.write("This is written in external file")
```

Understand the concept of document.write(),document.writeln(),console.log()

1) document.write()

- Writes output directly on the web page.
- Mostly used for learning.
- Can overwrite the page if used after load.

2) document.writeln()

- Same as document.write() but adds a newline.
- Newline is not visible in HTML unless
 is used.

3) console.log()

- Prints output in the browser console.
- Used for debugging.
- Does not display on the web page.

Example

```
<script>
document.writeln("Hello");
document.writeln("World");
</script>
```

Output on web page:

Hello World

What happens?

- document.writeln() adds a **newline in the source code**
- HTML ignores newlines

With
 (visible line break)

```
<script>
document.writeln("Hello<br>");
document.writeln("World");
</script>
```

Output:

Hello
World

ECMAScript (ES6) Overview

What is ECMAScript (European Computer Manufacturers Association Script)?

ECMAScript is the **standard or specification** that defines how JavaScript should work.

JavaScript is an **implementation of ECMAScript**.

In simple words:

ECMAScript = Rules

JavaScript = Language built using those rules

What is ES6?

ES6 (ECMAScript 2015) is a **major update** to JavaScript introduced in **2015**.

It added **modern, efficient, and cleaner syntax**.

Key Features of ES6

- let and const

- Arrow functions
- Template literals
- Classes
- Spread and Rest operators
- Default parameters

- ✓ JavaScript is a scripting language used for interactive web development.
- ✓ ECMAScript is the standard that defines JavaScript.
- ✓ ES6 is an updated version of ECMAScript with modern features.
- ✓ ES6 makes JavaScript code shorter, cleaner, and more efficient.
- ✓ Modern applications use ES6 by default.

Variables

Introduction to Variables

- Variables are **containers used to store data values**.
- They are the **basic building blocks** of any programming language.
- In JavaScript, variables store **reusable values**.
- Values are assigned using the **assignment operator (=)**.
- JavaScript is a **dynamically typed language**, so variable types are decided at **runtime**.

JavaScript Identifiers

JavaScript variables must have **unique names**, called **identifiers**.

Rules for Identifiers

- Must start with a **letter (a–z / A–Z)**, **underscore (_)**, or **dollar sign (\$)**
- Can contain digits after the first character (e.g., value1)
- Are **case-sensitive** (x and X are different)
- Cannot be **reserved keywords**

Ways to Declare Variables in JavaScript

JavaScript variables can be declared in **four ways**:

1. Implicit (Automatically)
2. Using var
3. Using let
4. Using const

Initially, only var was used.

In **ES6**, let and const were introduced for better control and safety.

Example:

```
x=10;  
var x =10;  
let x=10;  
const x=10
```

Local and Global scope

JavaScript local variable

- ✓ A JavaScript local variable is declared inside block or function.
- ✓ It is accessible within the function or block only.

For example:

```
<script>
function abc(){
var x=10;//local variable
}
abc();//calling JavaScript function
</script>
```

Output:

10

JavaScript global variable

- ✓ A global variable is declared outside functions and is accessible throughout the program, but its use should be minimized to avoid scope conflicts.

For example:

```
<script>
var d=200;//global variable
function a(){
document.writeln(d);
}
document.writeln(d);
a();//calling JavaScript function
</script>
```

Output:

200 200

Implicit (Automatically)

Implicit variable declaration occurs when a variable is assigned a value without using `var`, `let`, or `const`. Such variables become global by default, which can cause unexpected behavior, make debugging difficult, and lead to errors. Therefore, this practice should be avoided.

```
x = 5;  
y = 6;  
z = x + y;
```

var

- i. The **var** is the oldest keyword to declare a variable in JavaScript.
- ii. It has the **Global scoped** or **function scoped** which means variables defined outside the function can be accessed globally, and variables defined inside a particular function can be accessed within the function.

1. The below code example explains the use of the `var` keyword to declare the variables in JavaScript.

```
var a = 10  
function f1() {  
    var b = 20  
    console.log(a, b)  
}  
f1();  
console.log(a);
```

Output:

```
10 20  
10
```

2. The below example explains the behavior of `var` variables when **declared inside a function** and accessed outside of it.

```
function f1() {  
    // It can be accessible anywhere within this function  
    var a = 10;  
    console.log(a)  
}  
f1();
```

// A cannot be accessible outside of function

```
console.log(a);
```

Output:

10

ReferenceError: a is not defined

3. The below code redeclare a variable with same name in the same scope using the var keyword, which gives no error in the case of var keyword.

```
var a = 10
```

// User can re-declare variable using var

```
var a = 8
```

// User can update var variable

```
a = 7
```

```
console.log(a);
```

Output:

7

4. The below code explains the initialized the variables after accessing.

```
console.log(a);
```

```
var a = 10; //initialized the variable after accessing
```

Output:

Undefined

let

- ✓ The let keyword is an improved version of the var keyword.
- ✓ It is introduced in the ES6 or EcmaScript 2015.
- ✓ These variables has the **block scope**.
- ✓ It can't be accessible outside the particular code block ({block}).

1. The below code declares the variable using the let keyword.

```
let a = 10;
function f() {
  let b = 9
  console.log(b); //local scope
  console.log(a); //global scope
}
f();
console.log(b); // Error: b is not defined
```

Output:

```
9
10
Uncaught ReferenceError: b is not defined
```

Block scoped

```
function blockScopedExample() {
  var b=5
  if (b>4) {
    let a = 1; // Declared with let inside an if block
    console.log(a); // Output: 1 (Accessible inside the block)
  }
  console.log(a); // Error: a is not defined (Not accessible outside the block)
}
```

```
blockScopedExample();
```

Output:

```
1
ReferenceError: a is not defined
```

2. The below code explains the behavior of let variables when they are re-declared in the same scope.

```
let a = 10
// It is not allowed
let a = 10
// It is allowed
a = 10
console.log(a);
```

Output:

Uncaught **SyntaxError**:Identifier 'a' has already been declared

3.The below code explains the behavior of let variable when it is redeclared in the different scopes.

```
let a = 10
function f1() {
  let a = 9
  console.log(a) // It prints 9 (local)
}
f1()
console.log(a) // It prints 10 (global)
```

Output:

```
9
10
```

4. The below code explains the initialized the variables after accessing.

```
console.log(a);
let a = 10; //initialized the variable after accessing
```

Output:

Uncaught **ReferenceError**: Cannot access 'a' before initialization

const

- ✓ The const keyword has all the properties that are the same as the let keyword, except the user cannot update it and have to assign it with a value at the time of declaration.
- ✓ These variables also have the block scope.
- ✓ It is mainly used to create constant variables whose values cannot be changed once they are initialized with a value.

1.This code tries to change the value of the const variable (reassignment).

```
const a = 10;  
function f() {  
  a = 9  
  console.log(a)  
}  
f();
```

Output:

TypeError: **Assignment to constant variable.**

This does indeed throw a TypeError because you're trying to reassign a const variable.

2.The below code explains the behavior of const variables when they are redeclared in the same scope.

```
const a = 10  
// It is not allowed  
const a = 10  
// It is not allowed  
a = 10
```

Output:

Uncaught SyntaxError: **Identifier 'a' has already been declared**

3. No reassignment is happening two independent const variables (a) exist in different scopes.

```
const a = 10;  
function f() {  
  const a = 9  
  console.log(a)
```

```

}
f();
console.log(a);

```

Output:

```

9
10

```

This works without error because the `const a = 9;` inside the function creates a new variable in the function's local scope, shadowing the global `a`.

Note:

The variables declared with `var` and `let` are mutable that is their value can be changed but variables declared using `const` are immutable.

var	let	const
The scope of a var variable is functional or global scope.	The scope of a let variable is block scope.	The scope of a const variable is block scope.
It can be updated and re-declared in the same scope . <pre> var a=10; var a=20; a=30; let a=45; </pre>	It can be updated but cannot be re-declared in the same scope . <pre> let b=20; b=45; //not allowed let b=24; var b=33; </pre>	It can neither be updated or re-declared in same scope . <pre> const b=20; //not allowed b=45; const b=33; let b=24; var b=33; </pre>
It can be declared without initialization. <pre> var a; </pre>	It can be declared without initialization. <pre> let a; </pre>	It cannot be declared without initialization. <pre> const a=20; </pre>

var	let	const
It can be accessed without initialization as its default value is “undefined” .	It cannot be accessed without initialization otherwise it will give ‘referenceError’ .	It cannot be accessed without initialization, as it cannot be declared without initialization.

Datatypes

JavaScript provides different data types to store different kinds of values.

There are **two main types of data types in JavaScript**:

1. **Primitive Data Types**
2. **Non-Primitive (Reference) Data Types**

Dynamic Typing in JavaScript

- JavaScript is a **dynamically typed language**.
- You do **not** need to specify the data type explicitly.
- The JavaScript engine determines the data type at runtime.
- Variables are declared using **var**, **let**, or **const**.

Example

```
var a = 20;    // holding number  
var b = "LJU"; // holding string
```

Primitive Data Types

Primitive data types are **predefined, built-in data types**. They store **single values** and are **immutable**.

An **immutable** value **cannot be changed after it is created**.

- Any modification creates a **new value**, not a change to the original.
- In JavaScript, **all primitive data types are immutable**.

Types of Primitive Data Types

1) String

Represents a sequence of characters.

```
let x = "LJU"; // String
```

2) Number

Represents numeric values (integers and floating-point numbers).

```
let x1 = 24.00; // with decimals  
let x2 = 24;    // without decimals
```

3) Boolean

Represents logical values: **true** or **false**.

```
let x = 5;  
let y = 5;  
let z = 6;
```



```
(x == y); // true
(x == z); // false
```

Comparison Operators

- == → compares values after type coercion
- === → compares **both value and type** (strict comparison)

4) Undefined

A variable is declared but **not assigned** a value.

```
let name;
console.log(name); // undefined
```

5) Null

Represents **intentional absence of value**.

```
let name = null;
typeof null returns "object".
```

Difference Between undefined and null

Feature	undefined	null
Meaning	Declared but not assigned	Explicitly no value
Type	"undefined"	"object"
Use Case	Uninitialized variables	Intentional empty value

Example: typeof Operator Example

```
<script>
var a = undefined;
var b = "test";
var c = true;
var d = 5;
let e = null;

document.write(typeof(a)+"<br>");
document.write(typeof(b)+"<br>");
document.write(typeof(c)+"<br>");
document.write(typeof(d)+"<br>");
document.write(typeof(e)+"<br>");
</script>
```

Output:

undefined
string
boolean
number
object

Example: String and Number Combination Example

```
<script>  
var x = 16 + 4 + "xyz";  
var y = "xyz" + 16 + 4;  
  
document.write(x + "<br>");  
document.write(y);  
</script>
```

Output:

20xyz
xyz164

Non-Primitive (Reference) Data Types

- Store **multiple values**
- Stored by **reference**
- Created by the programmer
- Mutable

Mutable Meaning:

A mutable **value** can be changed after it is created.

- The original value is **modified directly**.
- In JavaScript, **non-primitive (reference) types are mutable**.

Types of Non-Primitive Data Types

1. **Object**
2. **Array**

1) Object

Objects store data in **key : value pairs**.

Syntax

```
const person = {  
  firstName: "xyz",  
  lastName: "abc"  
};
```

Example

```
<script>  
const person = {firstName:"xyz", lastName:"abc"};  
document.write(person.firstName+" ");  
document.write(person["lastName"]);  
</script>
```

Output:

xyz abc

2) Array

- Arrays store **multiple values**
- Index starts from **0**
- Written using **square brackets []**
- Arrays are special types of objects

Example

```
const fruits = ["grapes", "mango", "watermelon"];
```

Array Properties

Length

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
let length = fruits.length;
```

Accessing Elements

```
fruits[0];           // First element  
fruits[fruits.length - 1]; // Last element
```

When to Use Arrays vs Objects

- JavaScript does **not** support associative arrays
- Use **objects** when keys are **strings**
- Use **arrays** when keys are **numbers (indexes)**

Conditions and Loops

CONDITIONS

Conditions help your program **make decisions** based on true/false logic.

Condition	WHY we use it	WHEN to use it
if	To check a single condition	One decision needed
if...else	To choose between two paths	Yes / No situations
else if	To handle multiple conditions	More than two outcomes
switch	To match one value with many options	Fixed values (menu, roles, days)
ternary (? :)	Short decision making	Simple conditions

◆ if Statement

- To execute code **only when condition is true**
- When you have **one condition** to check

Example

```
let isLoggedIn = true;
if (isLoggedIn) {
  console.log("Welcome user");
}
```

◆ if...else

- To choose **one of two paths**
- When decision has **only two outcomes**

Example

```
let age = 16;

if (age >= 18) {
  console.log("Allowed");
} else {
  console.log("Not allowed");
}
```

◆ else if

To handle **multiple conditions**

When you have **more than two possibilities**

Example

```
let score = 82;
```

```
if (score >= 90) {  
  console.log("Grade A");  
} else if (score >= 75) {  
  console.log("Grade B");  
} else {  
  console.log("Grade C");  
}
```

◆ switch Statement

- Cleaner than many else if
- Easier to read for **fixed values**
- When checking **one variable** against many exact values

Example

```
let role = "admin";  
switch (role) {  
  case "admin":  
    console.log("Full access");  
    break;  
  case "editor":  
    console.log("Edit access");  
    break;  
  case "viewer":  
    console.log("Read only");  
    break;  
  default:  
    console.log("No access");  
}
```

Use break to stop execution. Use default as fallback

◆ Ternary Operator (? :)

- Short and clean code
- Simple condition with one line output

Example

```
let age = 20;  
let result = age >= 18 ? "Adult" : "Minor";  
console.log(result);
```

Avoid ternary for complex logic

LOOPS

Loop	Syntax	Used For	Best When
for	for(init; condition; step)	Repeat code fixed number of times	You know how many times
while	while(condition)	Repeat while condition is true	Count unknown
do...while	do { } while(condition)	Runs at least once	Must execute once
for...of	for (value of iterable)	Loop through values	Arrays, strings
for...in	for (key in object)	Loop through keys	Objects

Loop Examples

◆ for loop

Used when number of iterations is known.

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output:

1 2 3 4 5

◆ while loop

Runs while condition is true.

```
let i = 1;
```

```
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

◆ do...while loop

Runs **at least once**, even if condition is false.

```
let i = 6;  
do {  
  console.log(i);  
  i++;  
} while (i <= 5);
```

Output:

6

◆ for...of Loop

- Used to loop through iterable values
- Best for **arrays, strings**

Example

```
let fruits = ["apple", "banana", "orange"];
```

```
// for...of loop iterates over the VALUES of the array
```

```
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

Output

```
apple  
banana  
orange
```

◆ for...in Loop (with Objects)

- Used to loop through **keys (properties)** of an object
- Best for **objects**

Example

```
let user = {  
  name: "Sonali",  
  age: 25,  
  city: "Delhi"  
};
```

```
// for...in loop iterates over the KEYS of the object
```

```
for (let key in user) {  
  console.log(key + ": " + user[key]);  
}
```

Output

```
name: Sonali  
age: 25  
city: Delhi
```

◆ for...in Loop with Arrays

- for...in **can** work with arrays
- It iterates over **indexes (keys)**, not values
- Not recommended for arrays

Example

```
let students = ["ABC", "PQR", "XYZ"];  
// for...in loop iterates over the INDEXES (keys) of the array  
for (let i in students) {  
  // 'i' represents the index (0, 1, 2)  
  console.log(i);  
  // Access the value using the index  
  console.log(students[i]);  
}
```

Output

```
0  
ABC  
1  
PQR  
2  
XYZ
```

Why for...in Is NOT Preferred for Arrays

It loops over **indexes**, not values.

Can include **extra properties** if added to array.

Order is **not guaranteed**.

Arrays depend on **sequence and values**

Comparison of var and let (Overwrite Effect)

Case	Code using var	Output	Code using let	Output
Condition (if)	var x=10; if(true){ var x=3; } console.log(x);	3	let x=10; if(true){ let x=3; } console.log(x);	10
Loop (for)	var i=5; for(var i=0;i<2;i++){ console.log(i);} console.log(i);	0 1 2	let i=5; for(let i=0;i<2;i++){ console.log(i);} console.log(i);	0 1 5

Array Methods

Method	Syntax	What it Does	Returns
push()	array.push(item)	Adds element(s) to the end of an array	New array length
pop()	array.pop()	Removes the last element from an array	Removed element
map()	array.map(callback)	Creates a new array by transforming each element	New array
filter()	array.filter(callback)	Creates a new array with elements that pass a condition	New array
forEach()	array.forEach(callback)	Executes a function for each element	undefined
includes()	array.includes(value)	Checks if array contains a value	true or false
find()	array.find(callback)	Finds the first element that matches a condition	Element or undefined

Examples

◆ push()

Adds elements to the **end** of an array.

```
let fruits = ["apple", "banana"];
fruits.push("orange");
console.log(fruits);
// ["apple", "banana", "orange"]
```

◆ pop()

Removes the **last element** from an array.

```
let numbers = [10, 20, 30];
let removed = numbers.pop();
console.log(numbers); // [10, 20]
console.log(removed); // 30
```

◆ map()

Used to **transform each element** and return a new array.

```
let nums = [1, 2, 3, 4];
let squares = nums.map(num => num * num);
console.log(squares); // [1, 4, 9, 16]
```

Original array is **not changed**

◆ filter()

Returns a new array with elements that satisfy a condition.

```
let ages = [12, 18, 25, 14];  
let adults = ages.filter(age => age >= 18);  
console.log(adults); // [18, 25]
```

◆ **forEach()**

Runs a function for each element (does **not** return a new array).

```
let names = ["Ali", "Saroj", "Janak"];  
names.forEach(name => {  
  console.log("Hello " + name);  
});
```

Used for **logging, updating UI, side effects**

◆ **includes()**

Checks if a value exists in an array.

```
let colors = ["red", "green", "blue"];  
console.log(colors.includes("green")); // true  
console.log(colors.includes("yellow")); // false
```

◆ **find()**

Returns the **first element** that matches a condition.

```
let users = [  
  { id: 1, name: "Ali" },  
  { id: 2, name: "Saroj" },  
  { id: 3, name: "Janki" }  
];  
let user = users.find(u => u.id === 2);  
console.log(user); // { id: 2, name: "Saroj" }
```

Stops searching after first match

Situation	Use
Add item to list	push()
Remove last item	pop()
Modify every item	map()
Select some items	filter()
Perform action only	forEach()
Check if value exists	includes()
Find one matching item	find()

++ and -- Operators in JavaScript

Operator	Name	Meaning	When value is updated
++a	Pre-increment	Increases value by 1 before using it	Immediately
a++	Post-increment	Uses current value, then increases by 1	After use
--a	Pre-decrement	Decreases value by 1 before using it	Immediately
a--	Post-decrement	Uses current value, then decreases by 1	After use

Examples

Pre-Increment (++a)

```
let a = 5;  
let b = ++a;
```

Result:

- a = 6
- b = 6

Value of a is increased **before** assignment.

Post-Increment (a++)

```
let a = 5;  
let b = a++;
```

Result:

- a = 6
- b = 5

Value of a is used first, then incremented.

Pre-Decrement (--a)

```
let a = 5;  
let b = --a;
```

Result:

- a = 4
- b = 4

Value of a is decreased **before** assignment.

Post-Decrement (a--)

```
let a = 5;  
let b = a--;
```

Result:

- $a = 4$
- $b = 5$

Value of a is used first, then decreased.

Combined Example

```
let x = 3;  
let y = x++ + ++x;  
console.log(x, y);
```

Output:

5 8

Explanation:

- $x++ \rightarrow$ uses 3, then x becomes 4
- $++x \rightarrow$ increases to 5, then uses 5
- $y = 3 + 5 = 8$

Functions

What is a Function?

- A function is a **group of reusable code** that can be called anywhere in a program.
- It avoids **repetition of code** and improves **reusability**.
- Functions help divide a large program into **small and manageable parts**.
- A function must be **defined before it is called**.
- In JavaScript, a function is defined using the **function keyword**.
- Function names follow the same rules as variables (letters, digits, _, \$).
- Parameters (if any) are written inside **parentheses ()**.
- The function body is written inside **curly braces { }**.
- A function may take **input (parameters)** and return an **output (return value)**.

Syntax of a JavaScript Function

```
function functionName(parameter1, parameter2, ...) {  
    // function body  
}
```

Function Invocation (Calling a Function)

A JavaScript function can be executed in **three ways**:

1. **Function Triggered by an Event (Button Click)**

A function is executed when a **user performs an action**, such as clicking a button.

Example

```
<html>  
<head>  
<script>  
function showMessage() {  
    alert("Button Clicked!");  
}  
</script>  
</head>  
<body>  
<input type="button" value="Click Me" onclick="showMessage()">  
</body>  
</html>
```

The function showMessage() runs **when the button is clicked**.

2. Function Called Explicitly from JavaScript Code

A function is executed by calling it **directly from JavaScript code**.

Example

```
<html>
<body>
<script>
function add(a, b) {
    return a + b;
}

let result = add(5, 10);
document.write(result);
</script>
</body>
</html>
```

The function add() is called **explicitly** in the code.

3. Automatically Executed Function (Self-Invoking Function)

A self-invoking function runs **automatically without being called**.

Example

```
<html>
<body>
<script>
(function () {
    document.write("This function runs automatically");
})();
</script>
</body>
</html>
```

This function executes **immediately when the page loads**.

Example: Function Without Passing Parameters

```
<html>
<head>
<script>
function fun() {
    document.write("Hello! How are you?");
}
</script>
</head>

<body>
<input type="button" onclick="fun()" value="Click">
</body>
</html>
```

Example: Function With Passing Parameters

```
<html>
<head>
<script>
function fun(f_name, l_name) {
    document.write("Hello " + f_name + " " + l_name + "!");
}
</script>
</head>

<body>
<input type="button" onclick="fun('ABC','XYZ')" value="Click">
</body>
</html>
```

JavaScript return Statement

- The return statement is used to **send a value back** from a function.
- Once return is executed, the function **stops executing**.
- Code written after return is **not executed**.
- A function can return:
 - **Primitive values** (number, string, boolean)
 - **Objects** (array, object, function)

Example: Function with Return Value

```
<html>
```



```
<body>
<script>
function Product(a, b) {
    return a * b;
}
document.write(Product(6, 10));
</script>
</body>
</html>
```

Assigning a Function to a Variable (Function Expression)

```
<html>
<body>
<script>
var a = function(x, y) {
    return x + y;
};
let result = a(3, 5);
document.write(result);
</script>
</body>
</html>
```

Example: One Function Calling Another Function

```
<html>
<head>
<script>
function info(first, last) {
    var full = first + " " + last;
    return full;
}
function hello() {
    var result = info('abc', 'xyz');
    document.write(result);
}
</script>
</head>
<body>
<input type="button" onclick="hello()" value="info">
</body>
</html>
```

ES6 Functions

Arrow functions, introduced in ES6 (ECMAScript 2015), provide a more concise syntax for writing functions in JavaScript.

Syntax

```
Function name = (param1, param2, ..., paramN) => expression
```

- **Parameters:** Listed in parentheses. If there is only one parameter, the parentheses can be omitted.
- **Expression:** After the arrow (=>), if there's a single expression, it is implicitly returned.

Examples

1. Arrow Function with Multiple Parameters

```
const add = (a, b) => a + b;  
console.log(add(3, 4)); // Output: 7
```

2. Arrow Function with a Single Parameter

```
const square = x => x * x;  
console.log(square(5)); // Output: 25
```

OR

```
const square = (x) => x * x;  
console.log(square(5)); // Output: 25
```

3. Arrow Function with No Parameters

```
const sayHello = () => "Hello!";  
console.log(sayHello()); // Output: "Hello!"
```

4. Arrow Function with a Block Body

For functions with multiple statements, use a block body with curly braces {}. You must use an explicit return statement to return a value.

```
const multiply = (a, b) => {  
  const result = a * b;  
  return result;  
};  
console.log(multiply(2, 3)); // Output: 6
```

If the function has only one statement, and the statement returns a value, you can remove the brackets and the return keyword:

Arrow Functions Return Value by Default:

```
fun1 = () => "Hello World!";  
console.log(fun1())
```

Example 1

```
<script>  
const newfun = p => `val=${p}`;  
console.log(newfun(30));  
  
fun1 = (val) => "Hello " + val;  
console.log(fun1('world'))  
</script>
```

Example 2

```
<script>  
const add = (j,k) => `${j+k}`;  
console.log(`Addition = ${add(5,9)}`);  
</script>
```

Example 3

```
<script>  
const add = (j,k) =>  
{  
let ans = j+k;  
return ans;  
}  
console.log(`Addition = ${add(10,15)}`);  
</script>
```

Understand concept of below example

Case-1

The function doesn't explicitly return anything because there is no `return` statement. In JavaScript, when a function doesn't return a value explicitly, it implicitly returns `undefined`. `add(8, 9)` is called, but it returns `undefined`. So, `console.log` concatenates "Ans=" with `undefined`.

```
<script>
const add=(j,k)=> { ans=`${j+k}`; }
console.log("Ans="+add(8,9));
</script>
```

Output : **Ans=undefined**

Case-2

The function uses a concise arrow function syntax (without curly braces). In such cases, the value of the expression after the arrow (=>) is implicitly returned.

The expression `ans = `${j} + k`` is evaluated and returned.

```
<script>
const add=(j,k)=> ans=`${j+k}`;
console.log("Ans="+add(8,9));
</script>
```

Output : **Ans=17**

Case-3

The function uses the full arrow function syntax with curly braces {}.

Inside the function body, the return statement explicitly returns `ans = `${j} + k``.

```
<script>
const add=(j,k)=> { return ans=`${j+k}`; }
console.log("Ans="+add(8,9));
</script>
```

Output : **Ans=17**

Implicit means the return happens automatically without writing return, while **explicit** means the return value is clearly specified using the return keyword.

In arrow functions, when curly braces {} are used, the return keyword is required; when curly braces are not used, the expression value is returned automatically.

TEMPLATE LITERALS

Template Literals are a modern way to create strings in JavaScript. They were introduced in ES6 and provide more flexibility than normal strings. Template literals make it easier to work with variables, expressions, and multiline text.

Syntax Difference

Normal Strings

Normal strings use single quotes (' ') or double quotes (" ").

Example:

```
let message = "Hello World";
```

Template Literals

Template literals use backticks (` `).

Example:

```
let message = `Hello World`;
```

Variable Interpolation

Normal Strings

Variables must be joined using the + operator.

Example:

```
let name = "abc";
```

```
let text = "Hello " + name + "!";
```

Template Literals

Variables can be inserted directly using \${ }.

Example:

```
let name = "abc";
```

```
let text = `Hello ${name}!`;
```

Using Expressions

Template literals allow JavaScript expressions inside \${ }.

Example:

```
let a = 5;
```

```
let b = 3;
```

```
let result = `Sum is ${a + b}`;
```

Normal strings require extra brackets and + operators.

Multiline Strings

Normal Strings

Multiline text requires special characters like `\n`.

Example:

```
let text = "Hello\nHow are you?\nGoodbye";  
Console.log(text);
```

Template Literals

Multiline strings work naturally.

Example:

```
let text = `Hello  
How are you?  
Goodbye`;  
Console.log(text);
```

Creating HTML Content

Normal Strings

HTML strings require heavy concatenation.

Example:

```
let html = "<div>" +  
"<h1>Title</h1>" +  
"<p>Text</p>" +  
"</div>";  
document.write(html);
```

Template Literals

HTML is easier to read and write.

Example:

```
let html = `

<h1>Title</h1>  
<p>Text</p>  
</div>`;  
document.write(html);


```

Rest parameter

- ✓ The rest parameter is an improved way to handle function parameters, allowing us to more easily handle various inputs as parameters in a function.
- ✓ The rest parameter syntax allows us to represent an indefinite number of arguments as an array.
- ✓ With the help of a rest parameter, a function can be called with any number of arguments, no matter how it was defined.
- ✓ Rest parameter is added in ES2015 or ES6 which improved the ability to handle parameter.

Syntax:

//... is the rest parameter (triple dots)

```
function functionname(...parameters)
{
  statement;
}
```

Note: When ... is added with parameter then it is the rest parameter. It stores n number of parameters as an array.

Example: without rest parameter. (passing arguments more than the parameters)

```
function fun(a, b){
  console.log(a);
  console.log(b);
}
fun(1,2,3,4,5);
```

Output:

```
1
2
```

- ✓ In the above code, no error will be thrown even when we are passing arguments more than the parameters, but only the first two arguments will be evaluated.
- ✓ It's different in the case of the rest parameter. With the use of the rest parameter, we can gather any number of arguments into an array and do what we want with them.

Example using rest parameter

```
function fun(a, b, ...c){
  console.log(a);
  console.log(b);
```

```
console.log(c);
}
fun(1, 2);
fun(1, 2, 3, 6, 5);
```

Output:

```
1
2
[3,6,5]
```

Note: The rest parameter has to be the last argument, as its job is to collect all the remaining arguments into an array.

Example: In this example, we are using the rest parameter with some other arguments inside a function.

```
function fun(a, b, ...c) {
  console.log(` ${a} ${b}`); //abc xyz
  console.log(c); //[ 'pqr', 'mno', 'stu']
  console.log(c[0]); //pqr
  console.log(c.length); //3
  console.log(c.indexOf('pqr')); //0
  console.log(c.indexOf('abc')); //-1
}
fun('abc', 'xyz', 'pqr', 'mno', 'stu');
```

In the above code sample, we passed the rest parameter as the third parameter, and then we basically called the function fun() with five arguments the first two were treated normally and the rest were all collected by the rest parameter hence we get 'pqr' when we tried to access c[0] and it is also important to note that the rest parameter gives an array in return and we can make use of the array methods that the javascript provides us.

The rest parameter allows us to represent an indefinite number of arguments as an array. By using the rest parameter, a function can be called with any number of arguments.

```
<script>
function f(...args)
{ console.log(args); }
f(1,2,3,4,5,6,7,8,9,10);
</script>
Output:
[1,2,3,4,5,6,7,8,9,10]
```


Spread Operator

- ✓ The JavaScript spread operator (...) allows us to quickly copy all or part of an existing array or object into another array or object.
- ✓ **The Spread operator** allows an iterable to expand in places where zero plus arguments are expected.
- ✓ It is mostly used in the variable array where there is more than 1 value is expected. It allows us the privilege to obtain a list of parameters from an array.
- ✓ **The syntax of the Spread operator is the same as the [Rest parameter](#) but it works opposite of it.**

```
arr1 = [10, 42, 35];
arr2 = [14, 56];

// normally used expand method
let arr_test = [arr1, arr2];
console.log(arr_test); //[[10, 42, 35],[14, 56]]

// spread
arr = [...arr1, ...arr2];
console.log(...arr); //10 42 35 14 56
console.log(arr); //[10,42,35,14,56]
console.log(Math.min(arr)); //NaN
console.log('Using spread operator: ' + Math.min(...arr)); //10
console.log(`Sorted: ${arr.sort()}` ); //Sorted: 10,14,35,42,56
```

Default parameters

Function parameters with default values are initialized with default values if they contain no value or are undefined.

JavaScript function parameters are defined as **undefined** by default.

However, it may be useful to set a different default value. That is where default parameters come into play.

Syntax:

```
function name(parameters_n, parameter_n=value) {  
}
```

Example 1: If we multiply two numbers in below example without passing a second argument and without using the default parameter, the answer that this function will return is **NAN(Not a Number)**, since if we do not pass the second parameter, the function will multiply the first number with **undefined**.

```
function multiply(a, b) {  
  return a * b;  
}  
let num1 = multiply(5);  
console.log(num1);  
let num2 = multiply(5, 8);  
console.log(num2);
```

Output:

NaN

40

Example 2: If we do **not pass a number as the second argument** and take the **default parameter as the second parameter**, it will multiply the first number with the default parameter, and if we pass two numbers as parameters, it will multiply the first number with the second number.

```
function multiply(a, b = 2) {  
  return a * b;  
}
```

```
let num1 = multiply(5);
console.log(num1);
let num2 = multiply(5, 8);
console.log(num2);
```

Output:

10
40

Default argument must be filled from rear(right) side only

Example to understand above sentence.

```
<script>
defaultfun = (p=2,q)=>
{
return(p+q);
}
console.log(`Add =${defaultfun(30)}`);
</script>
```

Output:

NaN

Passed argument assigned to the **p** so here **q is undefined**. Answer is **NaN**

The simple solution is to have all the parameters with default values on the right (rear side).

Example: Write Es6 function to be called on button click. Take one division with some text and no decoration initially. Take font size and style as default argument and pass background color and text color while passing argument to function style should change on button click.

```
<html>
<head>
</head>
<body>
<script>
function fun(bg,tx,size="50px",fs="italic")
```

```
{
i=document.getElementById("d1");
i.style.color=tx;
i.style.fontStyle=fs;
i.style.backgroundColor=bg;
i.style.fontSize=size;
}
</script>
<div id="d1">Hello</div>
<input type="submit" onclick="fun('pink','blue')" value="CLICK HERE"/>
</body>
```

Pop up Boxes: Alert, Confirm, Prompt

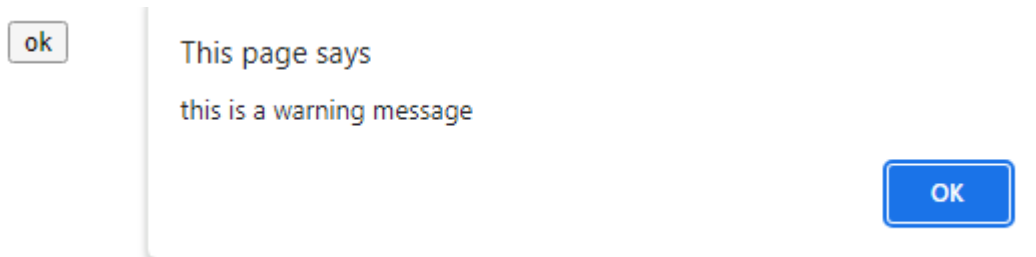
Alert Box

- ✓ An alert dialog box is mostly used to give a warning message to the users. For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.
- ✓ Nonetheless, an alert box can still be used for friendlier messages. Alert box gives only one button "OK" to select and proceed.

Example

```
<html>
<head>
<script>
function warn()
{
    alert("this is a warning message");
    document.write("Warning!!!!"); //It will be printed after closing alert box.//
}
</script>
</head>
<body>
    <input type="button" value="ok" onclick="warn()"/>
</body>
</html>
```

Output:



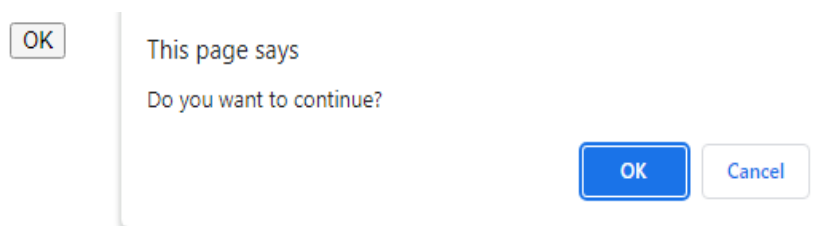
Confirmation Box

- ✓ A confirmation dialog box is mostly used to take user's consent on any option. It displays a dialog box with two buttons: OK and Cancel.
- ✓ If the user clicks on the OK button, the window method `confirm()` will return `true`. If the user clicks on the Cancel button, then `confirm()` returns `false`. You can use a confirmation dialog box as follows.

Example

```
<head>
<script>
function conf()
{
res = confirm("Do you want to continue?");  //It returns value as true/false
if(res==true)
{
    document.write("user wants");
    return true;
}
else
{
    document.write("user does not want");
    return false;
}
}
</script>
</head>
<body>
    <input type="button" onclick="conf()" value="OK"/>
</body>
```

Output:



Prompt Dialog Box

- ✓ The prompt dialog box is very useful when you want to pop-up a text box to get user input.
- ✓ Thus, it enables you to interact with the user. The user needs to fill in the field and then click OK.

This dialog box is displayed using a method called **prompt()** which takes two parameters:

- a label which you want to display in the text box and
- a default string to display in the text box.

Example:

```
<html>
<head>
<script type="text/javascript">
function getval()
{
    var res = prompt("Enter your name: ", "Your name here");
    document.write("you entered "+res);
}
</script>
</head>
<body>
<input type="button" onclick="getval()" value="OK"/>
</body>
</html>
```

Output:

